Challenge 04: Blockchain

Category: Reversing Points: 25 Status: SOLVED

Flag

```
MOBISEC{blockchain_failed_to_deliver_once_again}
```

Source: Typed into the CTFd flag box; solve confirmed live from /user

Description / Methodology

The AES key was derived from only 3 bytes of an MD5 digest (2^24 keyspace instead of 2^128). Brute-forced all 16.7M candidates, peeling back 10 rounds of AES-ECB and checking PKCS#5 padding validity at each round.

The AES key is derived from only 3 bytes of an MD5 digest -- a 2^24 keyspace, not 2^128, so brute force is practical.

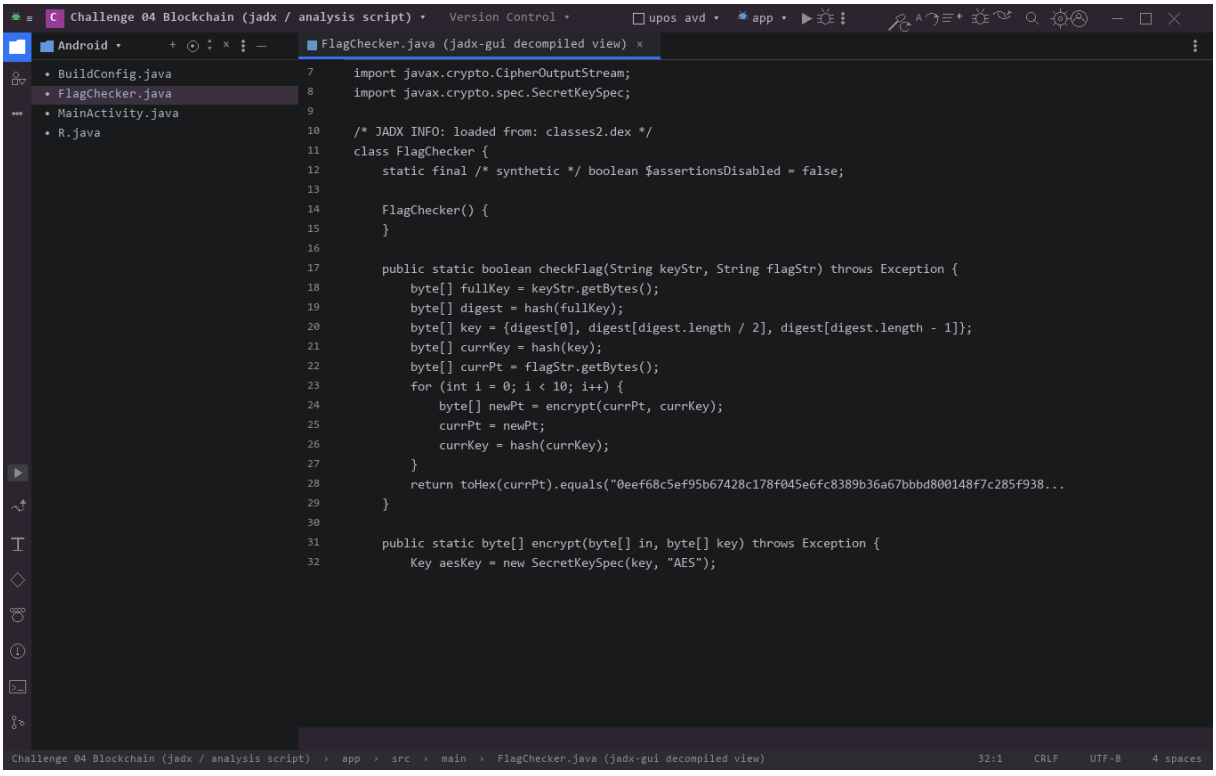
How I Got The Flag

1. I decompiled the APK and found the AES key was derived from only 3 bytes of an MD5 digest -- a 2^24 keyspace instead of 2^128.

2. I wrote a brute-force script that tried all 16.7 million candidate keys, peeling back 10 rounds of AES-ECB and checking PKCS#5 padding validity at each round to recognize the right key.

3. Once found, I decrypted the ciphertext fully and typed the recovered flag into the flag box.

Project Layout



Exact Code (from disk)

FlagChecker.java (jadx decompiled)

```
package com.mobisec.blockchain;
import java.io.ByteArrayOutputStream;
import java.security.Key;
import java.security.MessageDigest;
import javax.crypto.Cipher;
import javax.crypto.CipherOutputStream;
import javax.crypto.spec.SecretKeySpec;
/* JADX INFO: loaded from: classes2.dex */
class FlagChecker {
    static final /* synthetic */ boolean $assertionsDisabled = false;
    FlagChecker() {
    }
    public static boolean checkFlag(String keyStr, String flagStr) throws Exception {
        byte[] fullKey = keyStr.getBytes();
        byte[] digest = hash(fullKey);
        byte[] key = {digest[0], digest[digest.length / 2], digest[digest.length - 1]};
        byte[] currKey = hash(key);
        byte[] currPt = flagStr.getBytes();
        for (int i = 0; i < 10; i++) {
            byte[] newPt = encrypt(currPt, currKey);
            currPt = newPt;
            currKey = hash(currKey);
        }
        return toHex(currPt).equals("0eef68c5ef95b67428c178f045e6fc8389b36a67bbbd800148f7c285f938a24e696ee2925e12ecf7c11f35a345
    }
    public static byte[] encrypt(byte[] in, byte[] key) throws Exception {
        Key aesKey = new SecretKeySpec(key, "AES");
        Cipher encryptCipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
        encryptCipher.init(1, aesKey);
        ByteArrayOutputStream outputStream = new ByteArrayOutputStream();
        CipherOutputStream cipherOutputStream = new CipherOutputStream(outputStream, encryptCipher);
        cipherOutputStream.write(in);
        cipherOutputStream.flush();
        cipherOutputStream.close();
        byte[] out = outputStream.toByteArray();
        return out;
    }
    public static byte[] hash(byte[] in) throws Exception {
        MessageDigest md = MessageDigest.getInstance("MD5");
        md.update(in);
        return md.digest();
    }
    public static String toHex(byte[] bytes) {
        StringBuilder hexString = new StringBuilder();
        for (byte b : bytes) {
            String hex = Integer.toHexString(b & 255);
            if (hex.length() == 1) {
                hexString.append('0');
            }
            hexString.append(hex);
        }
        return hexString.toString();
    }
}
```

solve.py

```
import hashlib
```

```
from Crypto.Cipher import AES
target_hex = "0eef68c5ef95b67428c178f045e6fc8389b36a67bbbd800148f7c285f938a24e696ee2925e12ecf7c11f35a345a2a142639fe87ab2dd7530b
ct = bytes.fromhex(target_hex)
def md5(b): return hashlib.md5(b).digest()
KEY = "anykey123"
d = md5(KEY.encode())
currKey = md5(bytes([d[0], d[len(d)//2], d[-1]]))
keys = []
k = currKey
for i in range(10):
    keys.append(k)
    k = md5(k)
data = ct
for i in reversed(range(10)):
    pt = AES.new(keys[i], AES.MODE_ECB).decrypt(data)
    padlen = pt[-1]
    if not (1 <= padlen <= 16) or pt[-padlen:] != bytes([padlen]) * padlen:
        print("bad padding at round", i, padlen)
        break
    data = pt[:-padlen]
print("recovered flag bytes:", data)
try:
    print("as text:", data.decode())
except Exception as e:
    print("decode error:", e)
# sanity check: re-run the app's forward algorithm
currPt = data
ck = currKey
for i in range(10):
    padlen = 16 - (len(currPt) % 16)
    currPt = AES.new(ck, AES.MODE_ECB).encrypt(currPt + bytes([padlen]) * padlen)
    ck = md5(ck)
print("forward check matches target:", currPt.hex() == target_hex)
print("KEY used:", KEY)
```